Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра 09.02.07_Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА

Дисциплина(модуль)/предмет МДК 03.01 Внедрение и поддержка компьютерных систем

Тема: Проектирование безопасных релизных процессов, паттернов миграции данных и эксплуатационной документации ИС «Умный кампус»

ВАРИАНТ № 10 Спортивный комплекс (CAMPUS-110: изменение формата TimeSlot → TimeRange)

Студент (ка)/учащийся: Леонов Тимофей Андреевич /_____ /
Ф. И. О. подпись

Группа: И14-23-9

Руководитель: Чудинова А.А /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «22» октября 2026г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

1 ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА.....3
2 ЕДИНЫЙ ЦИФРОВОЙ СЛЕД (OPENTELEMETRY).....4
3 CHANGE REQUEST НА РЕЛИЗ V1.9.0.....6
4 ПРАВИЛА ЗАЩИТЫ ВЕТКИ MAIN (BRANCH PROTECTION).....8
5 МИГРАЦИЯ БД ПО ПАТТЕРНУ EXPAND AND CONTRACT.....9
6 PRODUCTION SANITY-ТЕСТ.....11
7 ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ (ADR, CHANGELOG).....12
ЗАКЛЮЧЕНИЕ.....13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....14

1 ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА

Вариант № 10. Модуль ИС «Умный кампус» — **«Спортивный комплекс»** (бронирование залов, бассейна, кортов). Задача Jira — **CAMPUS-110**. Суть миграции базы данных — изменение формата поля `TimeSlot` со строки (`string`) на структуру `TimeRange`.

Исходное состояние (версия 1.8.3): интервал брони хранится в колонке `sports.bookings.time_slot varchar(32)` в виде строки `2026-11-05 18:00-19:30`. Это приводит к двойному бронированию одной площадки (пересечение проверяется строками в коде, неатомарно), к путанице часовых поясов между мобильным приложением, сайтом и турникетами, к наличию двух форматов строки в базе и к невозможности индексировать поиск свободных окон.

Целевое состояние: в C# — value object `TimeRange(Start, End)` (полуоткрытый интервал [Start, End) в UTC), в PostgreSQL — встроенный диапазонный тип `tstzrange` с ограничением исключения `EXCLUDE USING gist`, которое физически запрещает пересекающиеся активные брони одной площадки.

Все исходные файлы размещены в публичном репозитории в папке `homework-5/`: https://github.com/kotokotru/smart-campus. Сервис реализован на ASP.NET Core 8, СУБД — PostgreSQL 16, тесты — xUnit, CI — GitHub Actions.

Блок задания	**Файл в репозитории**
1. Цифровой след	`src/Campus.Sports.Api/Controllers/SportsBookingsController.cs`, `Telemetry/CampusTelemetry.cs`
2. Change Management	`docs/change-requests/CR-2026-110-v1.9.0.md`, `docs/branch-protection.md`
3. Expand and Contract	`db/migrations/01_expand.sql`, `02_migrate.sql`, `03_contract.sql`, `Data/NpgsqlBookingRepository.cs`
4. Sanity и Docs-as-Code	`tests/Campus.Sports.Tests/ProductionSanityTests.cs`, `docs/adr/0009-timeslot-to-timerange.md`, `CHANGELOG.md`
Отчёт	`REPORT.md`, `report/` (PDF)

2 ЕДИНЫЙ ЦИФРОВОЙ СЛЕД (OPENTELEMETRY)

В контроллер `SportsBookingsController` добавлен проброс цифрового следа OpenTelemetry. Теги устанавливаются в текущий span HTTP-запроса `Activity.Current`, который создаёт ASP.NET Core instrumentation, и в дочерний доменный span `SportsBooking.Create` из собственного `ActivitySource("Campus.Sports")` [1, 2].

Листинг 1 — Фрагмент метода Create контроллера

```
[HttpPost]
public async Task<IActionResult> Create([FromBody] CreateBookingRequest request,
                                        CancellationToken ct)
{
    var bookingId = Guid.NewGuid();
    // 1) цифровой след в span HTTP-запроса
    CampusTelemetry.Enrich(Activity.Current, bookingId.ToString(), request.FacilityId);
    // 2) доменный дочерний span
    using var activity = CampusTelemetry.Source.StartActivity("SportsBooking.Create");
    CampusTelemetry.Enrich(activity, bookingId.ToString(), request.FacilityId);
    ...
    activity?.SetTag("campus.time_range.start", range.Start.ToString("O"));
    activity?.SetTag("campus.request.format",
        request.TimeSlot is null ? "time_range" : "legacy_time_slot");
```

Листинг 2 — Установка обязательных тегов (CampusTelemetry.Enrich)

```
public static void Enrich(Activity? activity, string? bookingId = null,
                          string? facilityId = null)
{
    if (activity is null) return;
    activity.SetTag("jira.issue", "CAMPUS-110");
    activity.SetTag("campus.db.migration_phase", "expand");
    if (bookingId  is not null) activity.SetTag("campus.booking_id",  bookingId);
    if (facilityId is not null) activity.SetTag("campus.facility_id", facilityId);
}
```

Обязательные теги: `jira.issue = CAMPUS-110` и доменный идентификатор сущности `campus.booking_id`. Дополнительно — `campus.facility_id`, `campus.time_range.start/end` и `campus.request.format`: по последнему тегу видно, сколько клиентов ещё присылают старый формат, и можно принять решение о фазе Contract. В `Program.cs` источник зарегистрирован через `AddOpenTelemetry().WithTracing(t => t.AddSource("Campus.Sports").AddAspNetCoreInstrumentation().AddOtlpExporter())`, в ресурс добавлены `service.version = 1.9.0` и SHA коммита.

Корректность установки тегов проверяет тест `TracingTagsTests`: через `ActivityListener` он перехватывает завершённые span’ы и проверяет, что и

доменный, и родительский HTTP-span содержат `jira.issue = CAMPUS-110` и тот же `campus.booking_id`, что вернул API.

Изменения зафиксированы Git-коммитом с сообщением `[CAMPUS-110] Add tracing tags and business logic` (`fb18213`, https://github.com/kotokotru/smart-campus/commit/fb18213d599477dba6bfda5d206b22241f308e78). Формат `[CAMPUS-NNN]` проверяется в CI (job `commit-lint`). В итоге формируется сквозная цепочка CALS-цифрового следа: задача Jira → коммит → CI → образ 1.9.0 → аннотация `kubernetes.io/change-cause` → span с `jira.issue` → запись в `public.schema_migrations`.

3 CHANGE REQUEST НА РЕЛИЗ V1.9.0

Карточка Change Request оформлена в файле `docs/change-requests/CR-2026-110-v1.9.0.md` [6, 7]. Основные поля приведены в таблице 1.

Таблица 1 — Карточка CR-2026-110

Поле	**Значение**
Номер / задача	CR-2026-110 / CAMPUS-110
Сервис	campus-sports-api, v1.8.3 → v1.9.0
Тип, риск	Normal Change (CAB), риск средний; ожидаемый простой — 0 мин
Описание	Ввод TimeRange/tstzrange вместо строки TimeSlot; фазы Expand и Migrate; API принимает и возвращает оба формата; теги OpenTelemetry; Sanity-тест
Бизнес-цель	Исключить двойное бронирование площадок (37 обращений за сентябрь → 0), единое время брони во всех клиентах, основа для поиска свободных окон (CAMPUS-118)
Окно обслуживания	Среда 28.10.2026, 03:00–05:00 МСК
Согласующие	Тимлид платформы, DBA, Product Owner модуля, руководитель спорткомплекса

Обоснование окна обслуживания. По данным мониторинга за 8 недель минимальный трафик сервиса — 03:00–05:00 (≈ 2 RPS против 180 RPS в пике 17:00–21:00). Спорткомплекс закрыт с 23:00 до 07:00, турникеты не работают. Ночной бэкап БД завершается в 02:30, поэтому к началу работ есть свежая резервная копия. До открытия в 07:00 остаётся два часа на откат. Среда выбрана потому, что в понедельник в 08:00 открывается запись на неделю (всплеск броней), а релиз в пятницу оставил бы проблему на выходные без дежурной команды.

План проведения работ:

1. 03:00 — фиксация текущей ревизии: `kubectl rollout history deployment/campus-sports-api -n campus-sports`.
2. 03:05 — фаза Expand: `psql -v ON_ERROR_STOP=1 -f 01_expand.sql` (DBA).
3. 03:10 — выкат: `kubectl set image deployment/campus-sports-api api=…:1.9.0` и аннотация change-cause.

4. 03:12 — контроль: `kubectl rollout status … --timeout=5m` (rolling update, maxUnavailable: 0).

5. 03:20 — Production Sanity-тест: `dotnet test --filter "Category=Sanity"`.

6. 03:25 — фаза Migrate: `02_migrate.sql`, проверка «NULL = 0, errors = 0, mismatches = 0».

7. 03:35 — проверка трейсов по `jira.issue=CAMPUS-110`; до 04:30 — наблюдение (5xx, p95, pg_locks).

8. 04:30 — закрытие CR, запись в CHANGELOG, оповещение в канале релизов.

План отката. Критерии: доля 5xx > 1 % в течение 5 минут, p95 > 800 мс, падение Sanity-теста или `rollout status` не завершился за 5 минут. Действия (целевое время ≤ 5 мин):

```
kubectl rollout undo deployment/campus-sports-api -n campus-sports
kubectl rollout status deployment/campus-sports-api -n campus-sports --timeout=5m
kubectl rollout history deployment/campus-sports-api -n campus-sports
```

Откат БД не требуется: после фазы Expand схема обратно совместима — версия 1.8.3 не знает о колонке `time_range`, а `time_slot` продолжает заполняться (dual-write и триггер). При проблеме в самой схеме DBA выполняет `01_expand_rollback.sql`. Точка невозврата наступает только в релизе v1.10.0 при выполнении `03_contract.sql`, который оформляется отдельным CR.

4 ПРАВИЛА ЗАЩИТЫ ВЕТКИ MAIN (BRANCH PROTECTION)

Конфигурация защиты ветки `main` описана в `docs/branch-protection.md` и продублирована в виде JSON для применения через GitHub API (`gh api -X PUT repos/{owner}/{repo}/branches/main/protection`) [3].

Таблица 2 — Branch Protection Rules для ветки main

Настройка	**Значение**
Require a pull request before merging	Да — прямой push в main запрещён
Required approvals	2 одобрения
Require review from Code Owners	Да — изменения db/migrations/ одобряет DBA
Dismiss stale approvals / Require approval of the most recent push	Да / Да
Require status checks to pass	commit-lint, build-and-test, db-migrations
Require branches to be up to date	Да
Require conversation resolution	Да
Require signed commits / linear history	Да / Да
Do not allow bypassing (включая админов)	Да
Allow force pushes / deletions	Нет / Нет

Обязательные проверки CI определены в `.github/workflows/homework-5-ci.yml`: `commit-lint` проверяет, что каждый коммит и заголовок PR начинаются с `[CAMPUS-NNN]`; `build-and-test` собирает решение и запускает все xUnit-тесты; `db-migrations` поднимает PostgreSQL 16 и последовательно применяет миграции 00 → 01 → 02 (дважды, проверка идемпотентности) → 03, а также проверяет откат фазы 1 на чистой базе. Для тегов `v*` действует защита тегов, а деплой в окружение production требует ручного одобрения тимлида и SRE.

5 МИГРАЦИЯ БД ПО ПАТТЕРНУ EXPAND AND CONTRACT

Миграция разделена на три фазы и два релиза, чтобы в каждый момент времени со схемой могли работать и старые (1.8), и новые (1.9) экземпляры приложения — без простоя [4, 5, 8].

Фаза 1 — Expand (01_expand.sql, v1.9.0). Добавляется колонка `time_range tstzrange` без значения по умолчанию — это изменение только каталога, выполняется за миллисекунды. CHECK-ограничение корректности интервала создаётся `NOT VALID` (без сканирования таблицы). Создаются функции `parse_legacy_time_slot` / `format_legacy_time_slot` и триггер синхронизации, который во время rolling update заполняет новую колонку для записей от подов 1.8. Индекс GiST по `(facility_id, time_range)` строится `CONCURRENTLY`. Установлен `lock_timeout = 3s`.

Листинг 3 — Фрагмент 01_expand.sql

```
SET lock_timeout = '3s';
ALTER TABLE sports.bookings ADD COLUMN IF NOT EXISTS time_range tstzrange;
ALTER TABLE sports.bookings ADD CONSTRAINT bookings_time_range_valid CHECK (
    time_range IS NULL OR (NOT isempty(time_range) AND lower_inc(time_range)
    AND NOT upper_inc(time_range) AND upper(time_range) - lower(time_range)
        BETWEEN interval '15 minutes' AND interval '4 hours')) NOT VALID;
CREATE TRIGGER trg_bookings_sync_time_slot BEFORE INSERT OR UPDATE OF time_slot, time_range
    ON sports.bookings FOR EACH ROW EXECUTE FUNCTION sports.bookings_sync_time_slot();
CREATE INDEX CONCURRENTLY IF NOT EXISTS ix_bookings_facility_time_range
    ON sports.bookings USING gist (facility_id, time_range) WHERE status = 'Active';
```

C#-код одновременно пишет данные в старую и новую структуру (dual-write) в одной команде INSERT; запись в старую колонку управляется флагом `Migration:WriteLegacyTimeSlot`:

Листинг 4 — Dual-write в NpgsqlBookingRepository.CreateAsync

```
var writeLegacy = migration.CurrentValue.WriteLegacyTimeSlot;
await using var insert = new NpgsqlCommand("""
    INSERT INTO sports.bookings
        (id, facility_id, student_id, time_range, time_slot, status, is_sanity_test, created_at)
    VALUES (@id, @f, @s, @r, @slot, @status, @sanity, @created)
    """, conn, tx);
insert.Parameters.Add(new NpgsqlParameter("r", NpgsqlDbType.TimestampTzRange) {      // НОВАЯ
    Value = new NpgsqlRange<DateTime>(range.Start.UtcDateTime, true,
                                      range.End.UtcDateTime, false) });
insert.Parameters.Add(new NpgsqlParameter("slot", NpgsqlDbType.Varchar) {          // СТАРАЯ
    Value = writeLegacy ? booking.LegacyTimeSlot : DBNull.Value });
```

Фаза 2 — Migrate (02_migrate.sql, v1.9.0). Процедура `sports.backfill_time_range` переносит исторические данные пакетами по 5 000 строк; каждый пакет — отдельная короткая транзакция (`COMMIT` внутри

процедуры), строки выбираются `FOR UPDATE SKIP LOCKED`, между пакетами — пауза. Нераспознанные строки записываются в журнал `sports.bookings_migration_errors`. После переноса выполняется проверка (нет NULL, нет ошибок, нет расхождений), затем создаётся `CHECK (time_range IS NOT NULL) NOT VALID` и выполняется `VALIDATE CONSTRAINT` — с блокировкой, не мешающей записи. Скрипт идемпотентен.

Фаза 3 — Contract (03_contract.sql, v1.10.0). Выполняется после перевода всех клиентов и 14 дней без запросов в старом формате. Скрипт проверяет предусловия, удаляет триггер, выполняет `SET NOT NULL` (без сканирования благодаря валидному CHECK), удаляет колонку `time_slot` и служебные функции, добавляет `EXCLUDE USING gist (facility_id WITH =, time_range WITH &&)`.

Проверка. Все скрипты выполнены на PostgreSQL 16 на демо-наборе из 46 002 броней (результаты в таблице 3); тот же сценарий автоматически повторяет CI.

Таблица 3 — Результаты прогона миграций

Шаг	**Результат**
01_expand.sql	0,35 с; запись от «старого» клиента без time_range — триггер заполнил интервал
02_migrate.sql (с ошибочной строкой)	46 002 строки перенесены за 10 пакетов; строка «завтра вечером» попала в журнал ошибок, скрипт остановился до VALIDATE
02_migrate.sql (после исправления)	NULL = 0, errors = 0, mismatches = 0; ограничения VALID; повторный запуск — без изменений
03_contract.sql	time_slot удалена, time_range NOT NULL; пересекающаяся бронь → ошибка exclusion constraint bookings_no_overlap
01_expand_rollback.sql	схема возвращена к версии 1.8, данные (46 002 строки) сохранены

6 PRODUCTION SANITY-ТЕСТ

Тест `CreateBooking_InTimeRangeFormat_ThenCleanup` класса `ProductionSanityTests` (xUnit, категория `Sanity`) выполняет точечную проверку боевого сценария сразу после релиза — создание тестовой брони:

1. создаётся бронь служебной площадки `sanity-court-00` в новом формате `start/end` на 15 минут в ночное время случайного дня через год и более — чтобы не мешать реальным пользователям; заголовок `X-Campus-Sanity: true` помечает её как тестовую;
2. проверяется `201 Created`, точное совпадение `start/end` и заполнение старого поля `timeSlot` (dual-write);
3. бронь читается через `GET` и сравнивается с ответом создания;
4. повторная бронь того же слота должна вернуть `409 Conflict` (проверка пересечения интервалов);
5. **Cleanup** в `DisposeAsync` выполняется всегда, даже при падении проверок: `DELETE` (sanity-бронь удаляется физически) и контроль, что `GET` возвращает `404`.

Листинг 5 — Обязательный шаг очистки

```
public async Task DisposeAsync()
{
    foreach (var id in _created)
    {
        using var delete = await target.Client.DeleteAsync($"api/sports/bookings/{id}");
        Assert.Equal(HttpStatusCode.NoContent, delete.StatusCode);
        using var check = await target.Client.GetAsync($"api/sports/bookings/{id}");
        Assert.Equal(HttpStatusCode.NotFound, check.StatusCode);
    }
}
```

Запуск после релиза — с переменной окружения `SANITY_BASE_URL` (адрес боевого стенда) и фильтром `--filter Category=Sanity`. Если переменная не задана, фикстура `SanityTarget` поднимает сервис in-process через `WebApplicationFactory`, поэтому тот же сценарий выполняется в CI на каждом PR.

Результат прогона `dotnet test`: всего тестов — 12, пройдено — 12.

7 ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ (ADR, CHANGELOG)

ADR-0009 (`docs/adr/0009-timeslot-to-timerange.md`) фиксирует архитектурное решение по варианту: контекст (двойные брони, часовые пояса, два формата строки, отсутствие индексов), принятое решение (`TimeRange` / `tstzrange`, exclusion constraint, миграция Expand and Contract в двух релизах, dual-write + триггер), рассмотренные альтернативы и последствия [9].

Таблица 4 — Альтернативы, рассмотренные в ADR

Вариант	**Итог**
A. Оставить строку и улучшить разбор в коде	Отклонён — не решает гонку и часовой пояс
B. Две колонки start_at / end_at	Отклонён — нет встроенной проверки пересечений
C. tstzrange + EXCLUDE USING gist	Выбран — пересечения запрещены на уровне БД, есть индекс
D. ALTER COLUMN TYPE за один релиз	Отклонён — ACCESS EXCLUSIVE блокировка, ломает клиентов 1.8

Отрицательные последствия: временное усложнение (две колонки, триггер, флаг) до v1.10.0 и точка невозврата при `DROP COLUMN` — поэтому фаза Contract вынесена в отдельный CR.

CHANGELOG.md ведётся по стандарту Keep a Changelog [10] и Semantic Versioning. Запись `[1.9.0] - 2026-10-28` содержит разделы Added (TimeRange, колонка time_range, dual-write, теги OpenTelemetry, sanity-тест), Changed, Deprecated (поле timeSlot), Fixed (двойное бронирование, сдвиг времени на 3 часа, формат DD.MM.YYYY) и Breaking Changes (в 1.9.0 — нет, релиз обратно совместим). В разделе `[Unreleased]` анонсированы breaking changes версии 1.10.0.

ЗАКЛЮЧЕНИЕ

В ходе практической работы для модуля «Спортивный комплекс» ИС «Умный кампус» (вариант 10, задача CAMPUS-110) спроектирован безопасный релиз изменения формата `TimeSlot → TimeRange`.

Реализован единый цифровой след: каждый запрос к API несёт теги `jira.issue` и `campus.booking_id`, а цепочка «задача — коммит — CI — релиз — трейс — миграция» прослеживается от начала до конца. Оформлен Change Request на релиз v1.9.0 с обоснованным окном обслуживания (среда, 03:00–05:00), пошаговым планом работ и планом отката через `kubectl rollout undo`, а также описаны правила защиты ветки `main` с обязательными проверками CI.

Миграция базы данных выполнена по паттерну Expand and Contract в три фазы: добавление новой структуры с dual-write в коде, пакетный перенос исторических данных с контролем и удаление старой структуры отдельным релизом. Все скрипты проверены на PostgreSQL 16: миграция проходит без блокировок записи, идемпотентна, обнаруживает некорректные данные и откатывается. Итоговая схема физически исключает двойное бронирование площадок.

Написан Production Sanity-тест с обязательной очисткой тестовых данных, составлены ADR-0009 и CHANGELOG по стандарту Keep a Changelog. Работа позволила закрепить навыки управления изменениями, безопасной эволюции схемы БД без простоя и ведения документации по принципу Docs-as-Code.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. OpenTelemetry. .NET — Instrumentation. – URL: https://opentelemetry.io/docs/languages/dotnet/instrumentation/ (дата обращения: 22.10.2026).
2. OpenTelemetry. Semantic Conventions — Attribute Naming. – URL: https://opentelemetry.io/docs/specs/semconv/general/naming/ (дата обращения: 22.10.2026).
3. GitHub Docs. About protected branches. – URL: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches (дата обращения: 22.10.2026).
4. PostgreSQL 16 Documentation. ALTER TABLE. – URL: https://www.postgresql.org/docs/16/sql-altertable.html (дата обращения: 22.10.2026).
5. PostgreSQL 16 Documentation. Range Types. – URL: https://www.postgresql.org/docs/16/rangetypes.html (дата обращения: 22.10.2026).
6. Kubernetes Documentation. Deployments — Rolling Back a Deployment. – URL: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/ (дата обращения: 22.10.2026).
7. ITIL 4. Change Enablement Practice. – URL: https://www.axelos.com/certifications/itil-service-management (дата обращения: 22.10.2026).
8. Sadalage P. Parallel Change (Expand and Contract). – URL: https://martinfowler.com/bliki/ParallelChange.html (дата обращения: 22.10.2026).
9. Nygard M. Documenting Architecture Decisions. – URL: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions (дата обращения: 22.10.2026).
10. Keep a Changelog 1.1.0. – URL: https://keepachangelog.com/ru/1.1.0/ (дата обращения: 22.10.2026).